Cukoo Search

Parameter Tuning for Support Vector Machine:

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_classification(
    n_samples=2000,
    n_features=6,
    n_informative=4,
    n_redundant=1,
    random_state=42
)

X = StandardScaler().fit_transform(X)

n = 5
eggs = 15
iterations = 2
Pa = 0.25
lower = 50
upper = 125

def fitness(x):
    model = SVC(kernel="rbf", C=x[0], gamma=x[1])
    return cross_val_score(model, X, y, cv=5).mean()

nests = np.random.uniform(lower, upper, (n, 2))
scores = np.array([fitness(x) for x in nests])

best = nests[np.argmax(scores)].copy()
best_score = max(scores)

for _ in range(iterations):
```

```
    best = nests[np.argmax(scores)].copy()
    best_score = max(scores)

    for _ in range(iterations):
        for _ in range(eggs):
            i = np.random.randint(n)
            step = np.random.normal(0, 1, 2)
            new = nests[i] + 0.1 * step
            new = np.clip(new, lower, upper)

            score = fitness(new)
            j = np.random.randint(n)

            if score > scores[j]:
                nests[j] = new
                scores[j] = score

        abandon = int(Pa * n)

        for i in range(abandon):
            nests[i] = np.random.uniform(lower, upper, 2)
            scores[i] = fitness(nests[i])

        best_index = np.argmax(scores)

        if scores[best_index] > best_score:
            best = nests[best_index].copy()
            best_score = scores[best_index]

    print("Best C:", best[0])
    print("Best Gamma:", best[1])
    print("Best Accuracy:", best_score * 100)
```

Output:

```
Best C: 52.01428540606022
Best Gamma: 55.7094793662265
Best Accuracy: 58.94999999999999
```